Universidad de Antioquia
Departamento de Ingenieria Electrónica y de Telecomunicaciones
Informatica II

Informe Desafio N°2

Esteban García Lopez

Juan Camilo García Herrera

1. Análisis del problema

1.1. Descripción del problema

El problema consiste en desarollar la copa del mundo utilizando programación orientada a objetos y memoria dinámica, ademas haciendo persistentes los datos.

1.2. Objetivo General

Desarrollar una inspiración de la copa del mundo que permita simular el juego entre los diferentes equipos, goles, partidos ganados, perdidos etc.

1.3. Objetivo específico

Utilizar programación orientada a objetos, memoria dinámica y manejo de archivos.

1.4. Requerimientos funcionales

- Carga / actualización de datos: Se requiere desarrollar los algoritmos requeridos a fin de leer y actualizar los datos desde/hacia el almacenamiento permanente.
- Conformación de los grupos para la etapa clasificatoria de grupos.
- Simulación de los partidos en cada etapa de la copa.
- Generación de estadísticas del torneo
- Medición del consumo de recursos

1.5. Requerimientos no funcionales

- Rendimiento optimo para el sistema
- Diseño intuitivo y experiencia de usuario

1.6. Restricciones

El sistema contiene las siguiente restricciones debido a la finalidad del aprendizaje.

- Uso de librerías externas para la medicion de memoria
- Uso de librerías STL
- Uso de sintaxis ANSI

1.7. Análisis de posibles soluciones

Se planteo de acuerdo al diagrama de clases, una estructura, que facilite la persistencia de los datos, para que se guarden al final; Un array dinamico del objeto equipo, el cual tiene un array dinámico del objeto jugador, ya que pertenece al equipo, diferente clases como partido y grupo para mostrar la relación que los equipos tienen, ya sea cuando están en un partido o no.

2. Diagrama de clases

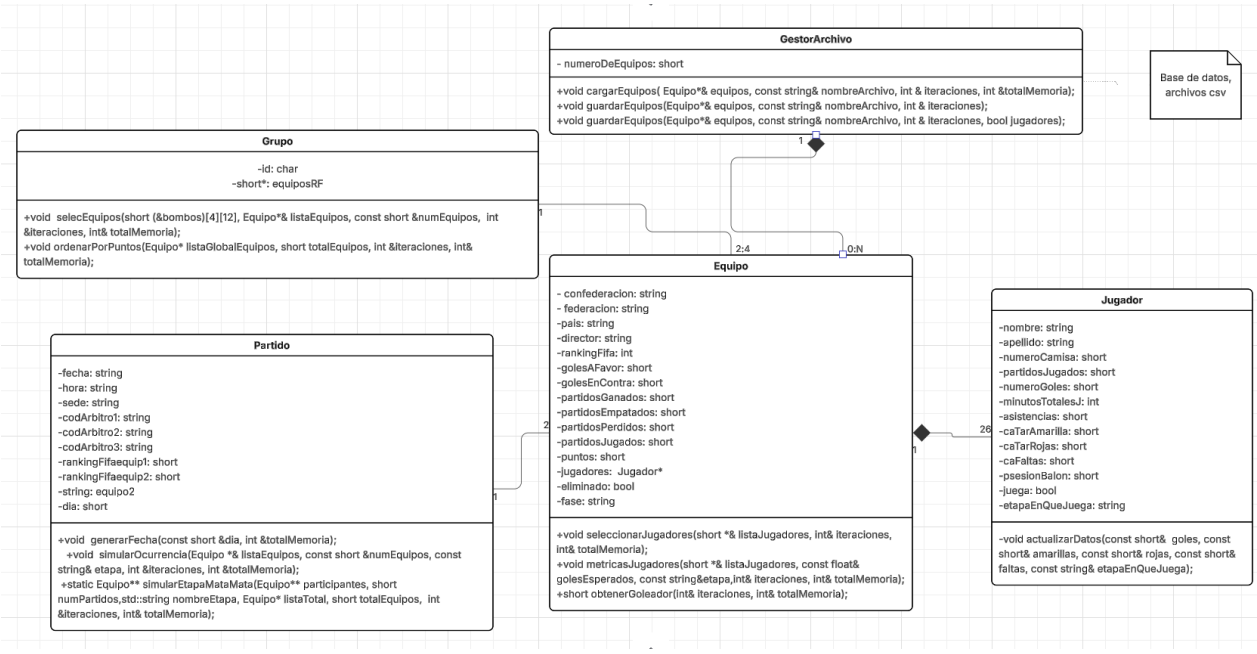

Figura 1: Diagrama de clases (Archivo adjunto en el repositorio)

2.1. Descripción

2.1.1. Equipo

- seleccionarJugadores(): método encargado de seleccionar 11 jugadores al hacer para cada partido
- metricasJugadores(): método encargado de generar los goles, faltas, tarjetas amarillas y rojas de cada jugador debido a la probabilidad que cada una de estas tiene.
- obtenerGoleador(): método encargado de obtener el jugador con mas goles.

2.1.2. Jugador

- actualizarDatos(): método encargado de actualizar los datos de cada jugador.

2.1.3. Partido

- generarFecha(): método que se encarga de generar la fecha de cada partido respetando la regla de los 3 días.
- simularOcurrencia(): este método se encarga de generar los goles esperados con base en la ecuación dada, así proceder a actualizar datos de los equipos.
- simularEtapaMataMata(): básicamente este "método"simula los partidos de 16, 8, 4 y finales, simulando la ocurrencia

2.1.4. Grupo

- selecEquipos(): método que selecciona los equipos teniendo en cuenta que no se pueden repetir confederaciones y generando que USA quede en el primer bombo.
- ordenarPorPuntos(): método que ordena los equipos según los puntos que ha alcanzado.

2.1.5. GestorArchivo

- cargarEquipos(): método que se encarga de traer todos los equipo del archivo dado, y entregarlo en una lista de equipos.
- guardarEquipo(): método que se encarga de guardar todos los datos finales de los equipos en un nuevo archivo.
- guardarEquipo(): sobrecarga de método que permite gaurdar en un archivo, los datos de todos los jugadores que han jugado.

3. Problemas de desarrollo

Debido a la extensa explicación del problema, a veces se olvidaba funcionalidades que pedían, por lo tanto había que regresar y editar código ya hecho consumiendo tiempo valioso.

Otro de los problemas de desarrollo que al que nos enfrentamos, fue la lógica de algunas funcionalidades; funcionalidades que hacían que el programa se dañara, haciéndonos buscar dónde está el problema.

4. Evolución de la solución

4.1. Modelado UML

Lo primero fue la creación de un **diagrama UML** de clases, lo que permitió visualizar y estructurar las entidades principales del sistema.

4.2. Desarrollo modular

Se comenzó con la implementación de módulos, dónde cada modulo es una clase, haciendo más legible el código y entendible.

4.3. Archivos

Se utilizó archivos .csv, debido a su facilidad de leer a la hora de abrirlo con excel.

5. Consideraciones a tener en cuenta en la implementación

5.1. Persistencia de datos

Se implementó un sistema de almacenamiento en .scv.

5.2. Tipos de datos

Se intentó hacer el sistema con los tipos de datos que aseguran un mayor rendimiendo debido a su poca almacenamiento en la memoria.